Arithmetic Circuits

Contemporary Digital Electronic Systems — Chapter 9

Muním Zabidi

Faculty of Electrical Engineering, Universiti Teknologi Malaysia

Table of Contents

1. Why This Chapter
2. Binary Arithmetic
3. Binary Adders
4. Ripple Carry Adders
5. Delay Analysis
6. Carry Look-Ahead Adders
7. Comparators
8. Parity
9. Summary

Why This Chapter Matters

Arithmetic is where number systems (Chapters 2–3) meet gate-level design (Chapters 4–8). Every ALU, address calculation and multiply-accumulate is built from the adders, subtractors and comparators in this chapter.

After this chapter, you can:

1. Design a half adder and a full adder at gate level.
2. Compose full adders into a ripple-carry adder of any width.
3. Convert an adder into a subtractor using two's complement.
4. Detect signed overflow in two's complement addition.
5. Explain how carry look-ahead shortens the carry chain.
6. Design a magnitude comparator.

© Muním Zabidi

An Opening Question

An 8-bit ripple-carry adder has a critical path of roughly 18 gate delays. How slow is a 64-bit one, and what would you do about it?

We derive the delay formula in the Delay Analysis section.

© Muním Zabidi

Table of Contents

1. Why This Chapter
2. **Binary Arithmetic**
3. Binary Adders
4. Ripple Carry Adders
5. Delay Analysis
6. Carry Look-Ahead Adders
7. Comparators
8. Parity
9. Summary

Notation and Binary Addition

- Lowercase (a_i, b_i) are single bits at position i; uppercase (A, B) are whole words.
- Add column by column from the LSB; a sum above 1 produces a carry.
- $1 + 1 = 10_2$: sum bit 0, carry 1.
- Adding two n-bit numbers may need $n + 1$ bits.

$a + b$	Carry	Sum	Decimal
0 + 0	0	0	0
0 + 1	0	1	1
1 + 0	0	1	1
1 + 1	1	0	2

Addition and Overflow (Unsigned)

$6 + 3$ in 4 bits

$$\begin{array}{cccc} & 0 & 1 & 1 & 0 \\ + & 0 & 0 & 1 & 1 \\ \hline & 1 & 0 & 0 & 1 \end{array} = 9 \checkmark$$

Carry out = 0: the answer is valid.

$10 + 7$ in 4 bits

$$\begin{array}{cccc} & 1 & 0 & 1 & 0 \\ + & 0 & 1 & 1 & 1 \\ \hline (1) & 0 & 0 & 0 & 1 \end{array} = 1 \, (\times)$$

Carry out = 1: the true sum, 17, does not fit.

- Discard the result or store the carry bit in a separate location.

Overflow in Two's Complement

- Unsigned: a carry out of the MSB signals overflow.
- Signed: the MSB is a sign, not a magnitude. Carry out can be 1 with no overflow, and overflow can occur with carry out 0.
- Overflow means the true result lies outside -2^{n-1} to $+(2^{n-1}-1)$.
- It occurs only when two operands of the same sign give a result of the opposite sign. Opposite-sign operands never overflow: the true result lies between them.

Operands	Sum	Reads as	True	Verdict
$0101 + 0100$	1001	-7	$+9$	Overflow: $+, + \to -$
$1110 + 1101$	1 1011	-5	-5	Valid, though carry out is 1
$0011 + 0100$	0111	$+7$	$+7$	Valid
$1000 + 1111$	1 0111	$+7$	-9	Overflow: $-, - \to +$

© Munim Zabidi

Detecting Overflow in Hardware

- Let c_{n-1} be the carry *into* the sign bit and c_n the carry *out of* it.
- Overflow occurs exactly when they differ:

$$V = c_{n-1} \oplus c_n$$

	c_{n-1}	c_n	V	Result
$0101 + 0100$	1	0	1	Overflow
$1110 + 1101$	1	1	0	Valid
$0011 + 0100$	0	0	0	Valid
$1000 + 1111$	0	1	1	Overflow

- Both carries already exist in the adder: detection costs one XOR gate.
- Processors keep separate flags: *C* for unsigned, *V* for signed. The same adder computes both.

Binary Subtraction

$a - b$	Borrow	Diff	Decimal
$1 - 0$	0	1	1
$1 - 1$	0	0	0
$0 - 0$	0	0	0
$0 - 1$	1	1	-1

Only $0 - 1$ is nontrivial: borrow from the next column.

$5 - 2$ (4 bits)

$0101 - 0010 = 0011 = 3$ ✓

$2 - 5$ (4 bits)

$0010 - 0101 = (1)1101$: a final borrow means the true result is negative and cannot be shown in unsigned binary.

Subtraction by Two's-Complement Addition

- Instead of separate hardware, convert the subtrahend to its two's complement and add: one adder does both jobs.

$5 - 2$ with 4 bits

Two's complement of 2 is 1110 ($= 2^4 - 2$).

$$\begin{array}{cccccc} & 0 & 1 & 0 & 1 & \\ + & 1 & 1 & 1 & 0 & = 3 \checkmark \text{ The carry out is discarded.} \\ \hline (1) & 0 & 0 & 1 & 1 & \end{array}$$

Binary Multiplication

- Two steps: generate partial products, then add them.
- Each multiplier bit is 0 (all-zero partial product) or 1 (copy of the multiplicand), shifted to its bit position.
- An m-bit by n-bit unsigned product fits in $m + n$ bits.

7×5

			1	1	1		(7)
$\times$			1	0	1		(5)
			1	1	1		
		0	0	0			
	1	1	1				
	1	0	0	0	1		1 (35)

Binary Division

- Shift and subtract, as in long division.
- If the subtraction is non-negative, enter 1 in the quotient and keep the result; otherwise enter 0 and restore.
- Division is slow and complex in hardware; it is often replaced by multiplication by the reciprocal.

$31 \div 5$

$11111 \div 101$:
$111 \geq 101$: quotient bit 1
$101 \geq 101$: quotient bit 1
$001 < 101$: quotient bit 0

Quotient $110 = 6$, remainder 1.

Table of Contents

1 Why This Chapter

2 Binary Arithmetic

3 **Binary Adders**

4 Ripple Carry Adders

5 Delay Analysis

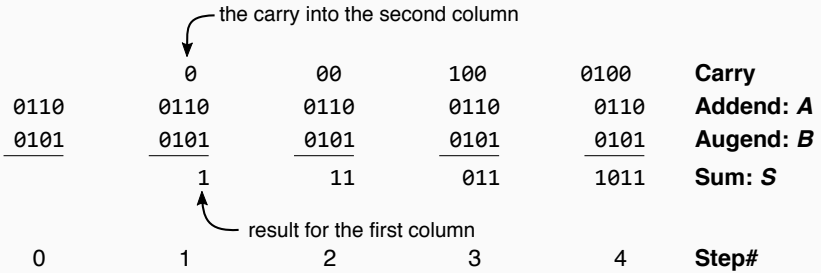
6 Carry Look-Ahead Adders

7 Comparators

8 Parity

9 Summary

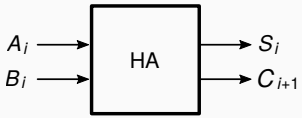
Adding Column by Column



- The LSB column adds two bits. Every other column adds three: a_i, b_i and the carry-in c_i.
- Each column produces a sum s_i and a carry c_{i+1} for the next column.

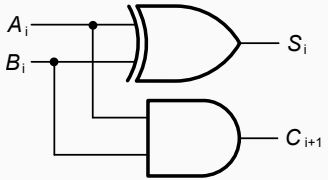
Half Adder

a_i	b_i	c_{i+1}	s_i
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



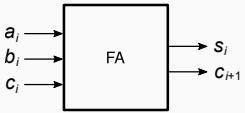
- Adds two bits: sum 0 to 2, so two outputs.
- It has no carry-in, so it works only at the LSB.

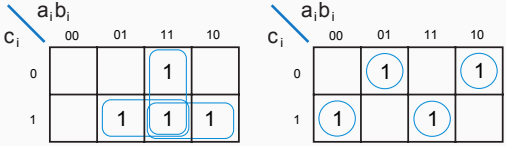
$$c_{i+1} = a_i b_i \qquad s_i = a_i \oplus b_i$$



Full Adder

a_i	b_i	c_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1



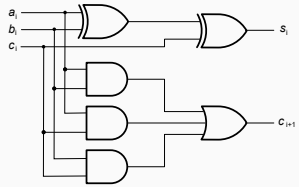


$c_{i+1} = a_ib_i + b_ic_i + a_ic_i$ $s_i = a_i \oplus b_i \oplus c_i$

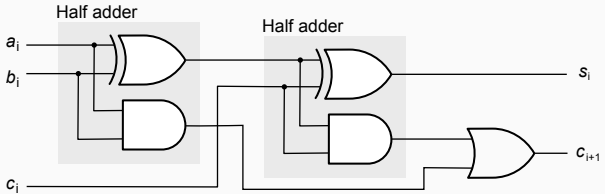
- Three inputs: the carry-in lets full adders be cascaded.

© Munim Zabidi

Full Adder Circuits



Two-level



Multi-level

- Multi-level: $c_{i+1} = a_i b_i + (a_i \oplus b_i) c_i$. It is faster when full adders are cascaded.
- $(a_i \oplus b_i) c_i$ may replace $(a_i + b_i) c_i$: when $a_i = b_i = 1$ the other term $a_i b_i$ already forces $c_{i+1} = 1$.

Table of Contents

1 Why This Chapter

2 Binary Arithmetic

3 Binary Adders

4 **Ripple Carry Adders**

5 Delay Analysis

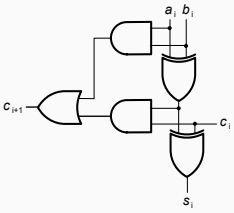
6 Carry Look-Ahead Adders

7 Comparators

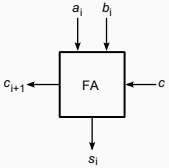
8 Parity

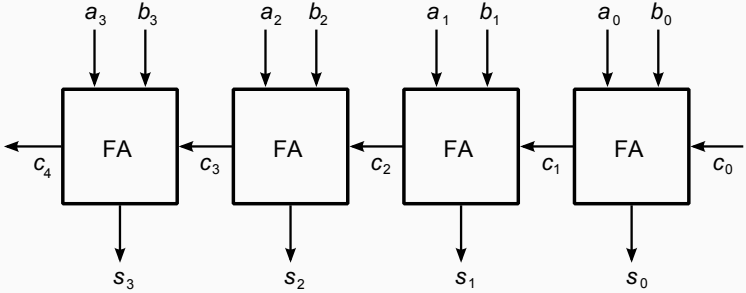
9 Summary

Ripple-Carry Adder



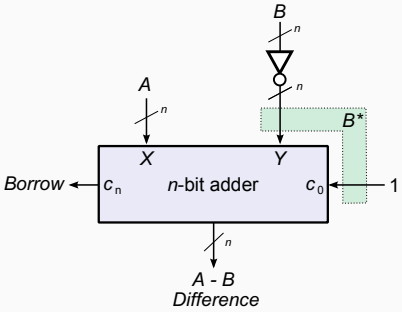
Cascadable full adder





- n full adders in sequence; stage 0 has $c_0 = 0$ (a half adder would also work, but full adders keep the design uniform).
- The carry ripples from stage to stage, so each adder waits for the previous one.
- Simple and cheap, but slow: fine where speed is not critical.

Subtraction with an Adder



- Invert the B inputs and set the carry-in to 1:

$$S = A + \overline{B} + 1 = A + B^* = A + (2^n - B) = A - B + 2^n$$

- The extra 2^n is the discarded carry, so $S = A - B$.
- Works with any adder: no dedicated subtractor circuit is needed.

© Munim Zabidi

Table of Contents

1 Why This Chapter

2 Binary Arithmetic

3 Binary Adders

4 Ripple Carry Adders

5 **Delay Analysis**

6 Carry Look-Ahead Adders

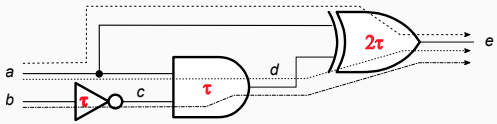
7 Comparators

8 Parity

9 Summary

Critical Path

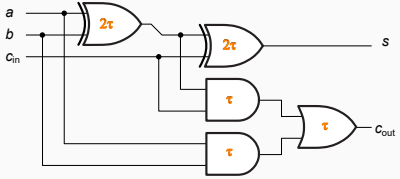
- The critical path is the path with the longest delay. It sets the circuit's maximum speed.
- Model: τ = delay of one gate; XOR = 2τ, other gates τ.



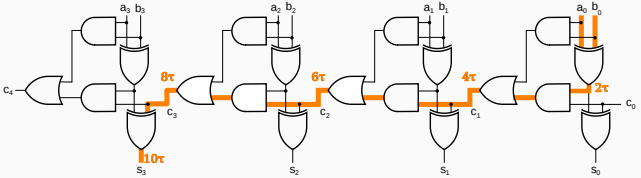
Path	Delay
$a \to e$	2τ
$a \to d \to e$	3τ
$b \to c \to d \to e$	4τ

Critical path delay: 4τ.

Delay of a Ripple-Carry Adder



- $a_i, b_i \rightarrow c_{i+1}$: 4τ
- $a_i, b_i \rightarrow s_i$: 4τ
- $c_i \rightarrow c_{i+1}$: 2τ
- $c_i \rightarrow s_i$: 2τ



The critical path runs from a_0/b_0 to s_{n-1}: 4τ for the first carry, 2τ for each of $n-2$ middle stages, 2τ for the last sum.

$$t_{pd} = [4 + 2(n-2) + 2]\tau = (2n+2)\tau$$

How slow is a 64-bit ripple adder?

$t_{pd} = (2n + 2)\tau$. For $n = 64$: 130τ, about seven times the 18τ of an 8-bit adder (a 16-bit adder takes 34τ).

The delay grows linearly with n. To fix it, we must speed up the carry chain. Three well-known techniques are carry look-ahead, carry-select and carry-save; this chapter covers carry look-ahead.

© Muním Zabidi

Table of Contents

1 Why This Chapter

2 Binary Arithmetic

3 Binary Adders

4 Ripple Carry Adders

5 Delay Analysis

6 **Carry Look-Ahead Adders**

7 Comparators

8 Parity

9 Summary

Generate and Propagate

- A CLA computes c_{i+1} *without waiting* for c_i, using a two-level circuit.
- Carry-out is 1 when $a_i = b_i = 1$ (generate), or when one input is 1 and the carry-in is 1 (propagate).

$$g_i = a_i b_i \qquad p_i = a_i + b_i$$

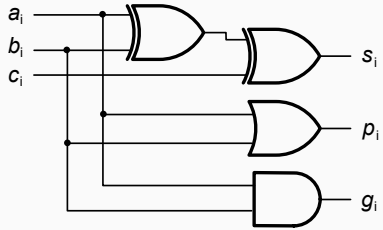
$$c_{i+1} = g_i + p_i c_i$$

a_i	b_i	c_i	c_{i+1}	g_i	p_i
Carry kill					
0	0	0	0	0	0
0	0	1	0	0	0
Carry propagate					
0	1	0	0	0	1
0	1	1	1	0	1
1	0	0	0	0	1
1	0	1	1	0	1
Carry generate					
1	1	0	1	1	1
1	1	1	1	1	1

Look-Ahead Carry Equations

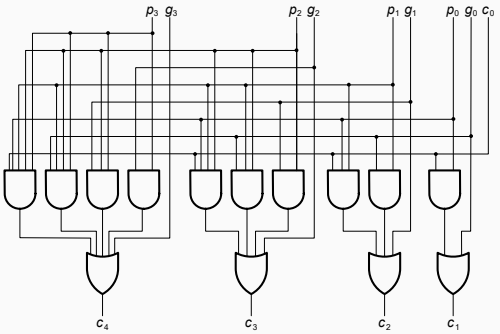
$$
\begin{aligned}
c_1 &= g_0 + p_0 c_0 \\
c_2 &= g_1 + p_1 g_0 + p_1 p_0 c_0 \\
c_3 &= g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0 \\
c_4 &= g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 \\
&\quad + p_3 p_2 p_1 p_0 c_0
\end{aligned}
$$

Each c_i depends only on c_0, not on c_{i-1}.

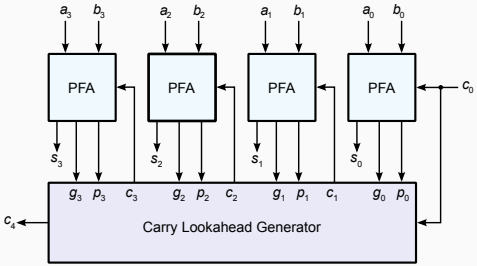


Partial full adder (PFA): s_i, p_i, g_i

4-Bit Carry Look-Ahead Adder



Carry look-ahead generator

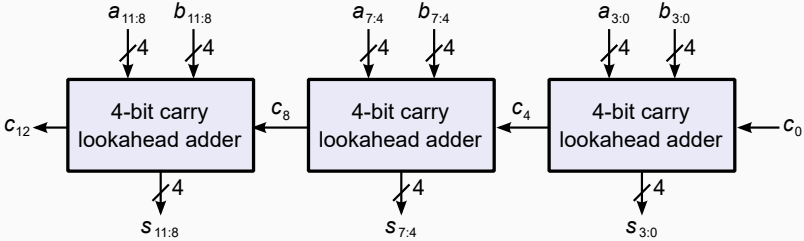


One generator and four partial full adders

$$t_{pd,\mathrm{CLA4}} = [1 + 2 + 2]\tau = 5\tau$$

1τ for p_i, g_i; 2τ for c_4 from c_0; 2τ for s_3 from c_3.

Cascading CLA Blocks



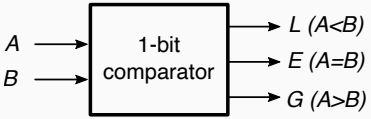
- A single CLA is very fast but needs much hardware, especially beyond 4 bits. Larger adders cascade CLA4 blocks.
- With m blocks: $t_{pd} = [1 + 2m + 2]\tau = (2m + 3)\tau$.
- 12 bits (3 blocks): 9τ, versus 26τ for a 12-bit ripple adder.

Table of Contents

1. Why This Chapter
2. Binary Arithmetic
3. Binary Adders
4. Ripple Carry Adders
5. Delay Analysis
6. Carry Look-Ahead Adders
7. **Comparators**
8. Parity
9. Summary

- Compare the MSBs first. If they differ, stop; if equal, move to the next bit.
- Equality comparators: output when two words are equal (e.g. a security lock).
- Magnitude comparators: $A < B, A = B, A > B$.
- Comparator ICs for 4 and 8 bits can be cascaded for bigger words.

A	B	L	E	G
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0



$$L = \overline{A}B,\ E = \overline{A \oplus B},\ G = A\overline{B}$$

Table of Contents

1 Why This Chapter

2 Binary Arithmetic

3 Binary Adders

4 Ripple Carry Adders

5 Delay Analysis

6 Carry Look-Ahead Adders

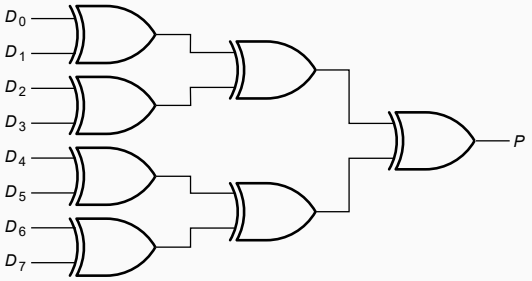
7 Comparators

8 **Parity**

9 Summary

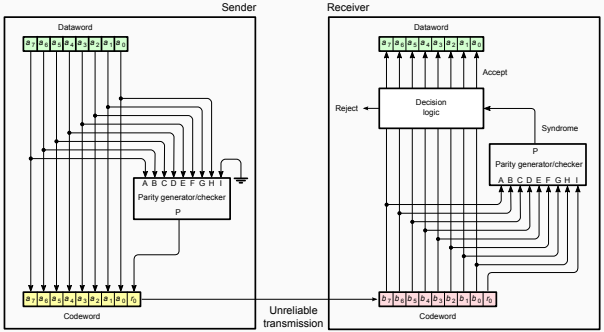
Parity Generator

- Parity detects single-bit errors. Even parity: the number of 1s in data plus parity bit is even.
- $P = D_7 \oplus D_6 \oplus \cdots \oplus D_0$: $P = 1$ when the data has an odd number of 1s.
- 'a' = 1100001 has three 1s, so $P = 1$; codeword 11100001.



© Muním Zabidi

Parity Checker



- The receiver computes the syndrome $S = P \oplus D_7 \oplus \cdots \oplus D_0$.
- $S = 0$: no error detected. $S = 1$: reject the byte. Changing 11100001 to 11100000 gives $S = 1$.
- An even number of bit errors is not detected, and parity cannot correct errors.

Table of Contents

1. Why This Chapter
2. Binary Arithmetic
3. Binary Adders
4. Ripple Carry Adders
5. Delay Analysis
6. Carry Look-Ahead Adders
7. Comparators
8. Parity
9. **Summary**

Summary

- A half adder adds two bits; a full adder also takes a carry-in. Full adders chain into an n-bit ripple-carry adder.
- Ripple-carry delay grows linearly with n; carry look-ahead shortens the carry chain with generate and propagate signals.
- Signed overflow: $V = c_{n-1} \oplus c_n$. Carry out alone is not an overflow indicator for two's complement.
- Two's-complement subtraction: add the inverted operand with carry-in 1, so one adder does both.
- A magnitude comparator gives $A < B$, $A = B$, $A > B$ by comparing from the most significant end.
- Parity bits detect single-bit errors but cannot correct them.

Looking ahead

Chapter 10 adds the missing ingredient, *memory*, with latches and flip-flops.